

UNIVERSITAS SEBELAS MARET - SEKOLAH VOKASI
PROGRAM STUDI D3 TEKNIK INFORMATIKA
MODUL PRAKTIKUM PEMROGRAMAN FRONT - END

**BAB VI: META-FRAMEWORK & SERVER-SIDE RENDERING (NEXT.JS APP ROUTER
& REACT SERVER COMPONENTS)**

1. Deskripsi Singkat Mata Kuliah

Mata kuliah Pemrograman Front End pada Program Studi D3 Teknik Informatika Sekolah Vokasi Universitas Sebelas Maret (UNS) dirancang untuk membekali mahasiswa dengan keterampilan rekayasa perangkat lunak antarmuka web modern berskala industri. Dalam era arsitektur web modern tahun 2026, pengembangan aplikasi web telah mengalami evolusi dari Single Page Applications (SPA) sisi klien murni menuju arsitektur Meta-Framework terintegrasi yang memanfaatkan Server-Side Rendering (SSR) dan React Server Components (RSC).

Modul praktikum bab ini berfokus pada penguasaan kerangka kerja Next.js App Router (direktori `app/`), arsitektur React Server Components (RSC), pemisahan tegas antara Server Components dan Client Components ("use client"), manajemen alur dynamic layout, streaming SSR dengan Suspense, middleware proteksi rute, hingga penerapan Metadata API untuk optimasi Search Engine Optimization (SEO) dan pencapaian indikator Core Web Vitals (LCP, INP, CLS). Melalui bab ini, mahasiswa dilatih untuk menyusun fondasi aplikasi web full-stack berskala enterprise yang tangguh, aman, dan berperforma tinggi.

2. Tujuan Praktikum & Capaian Pembelajaran (CPMK)

Setelah menyelesaikan bab praktikum ini, mahasiswa D3 Teknik Informatika UNS diharapkan menguasai kompetensi utama berikut:

- **Arsitektur File-Based Routing:** Memahami dan mengimplementasikan struktur arsitektur Next.js App Router menggunakan konvensi file direktori `app/` (`page.tsx`, `layout.tsx`, `loading.tsx`, `error.tsx`, `route.ts`).

Server vs Client Components: Menganalisis dan menerapkan perbedaan mendasar antara React Server Components (RSC) dan Client Components ("use client") untuk merender komponen di server guna meminimalkan ukuran bundle JavaScript klien.

Dynamic Layouts & Navigasi: Menyusun struktur layout bersarang (nested layouts) dan alur navigasi dinamis tanpa memicu pembatalan state atau re-render yang tidak perlu.

Data Fetching & Streaming SSR: Mengimplementasikan pola pengambilan data langsung di sisi server (Server-Side Data Fetching) serta memanfaatkan mekanisme caching, revalidasi dinamis (revalidate), dan Streaming SSR dengan React Suspense.

Metadata API & Core Web Vitals: Mengkonfigurasi Metadata API dinamis dan statis untuk optimasi SEO serta menerapkan teknik pemuatan aset terbaik guna memenuhi ambang batas Core Web Vitals (LCP $\leq$ 2.5s, INP $\leq$ 200ms, CLS $\leq$ 0.1).

Middleware & Server Security: Membangun lapisan kebiasaan middleware (middleware.ts) untuk manipulasi header, proteksi rute halaman berbasis autentikasi, dan pengalihan (redirect) sebelum permintaan mencapai komponen halaman.

Penerapan Proyek SRS Mandiri: Menerjemahkan Spesifikasi Kebutuhan Perangkat Lunak (SRS) proyek mahasiswa ke dalam arsitektur komponen Next.js yang terstruktur, modular, dan type-safe.

3. Dasar Teori Komprehensif

A. Paradigma Meta-Framework & Server-Side Rendering (SSR)

Perkembangan aplikasi web dalam dekade terakhir ditandai dengan pergeseran besar dari aplikasi web tradisional (Multi-Page Applications/MPA) menuju Single Page Applications (SPA) berbasis JavaScript seperti React, Vue, dan Svelte. Meskipun SPA menawarkan pengalaman pengguna yang halus tanpa reload halaman, SPA tradisional memiliki kelemahan mendasar: ketergantungan penuh pada eksekusi JavaScript di browser klien, ukuran bundle JavaScript yang membengkak, rentan terhadap masalah indeksasi Search Engine Optimization (SEO), serta nilai Time-to-First-Byte (TTFB) dan Largest Contentful Paint (LCP) yang buruk.

Untuk mengatasi hambatan tersebut, industri web mengadopsi Meta-Framework—kerangka kerja tingkat tinggi yang berdiri di atas pustaka UI (seperti React) yang secara otomatis menangani routing, server-side rendering (SSR), static site generation (SSG), bundling, dan optimasi aset di build-time maupun request-time (CloudRevol, 2026). Next.js yang dikembangkan oleh Vercel (dan kini didukung oleh ekosistem React Foundation) telah menjadi standar de-facto meta-framework React dengan adopsi mencapai 59% hingga 83.6% di kalangan praktisi web global (InfoQ, 2026; SVAR UI, 2026).

B. Arsitektur Next.js App Router (app/ Directory vs Legacy pages/ Router)

Sejak diperkenalkannya Next.js 13 dan pemantapannya pada versi 14 hingga 16, Vercel merombak total sistem routing dari konvensi direktori pages/ (Legacy Pages Router) menjadi direktori app/ (App Router) (CloudRevol, 2026). Perubahan ini bukan sekadar pergantian nama folder, melainkan fondasi ulang arsitektur pemrosesan halaman berbasis React Server Components (RSC).

Kriteria Evaluasi

Legacy Pages Router (pages/)

Modern App Router (app/)

Arsitektur Rendering	Sisi klien secara default, SSR via <code>getServerSideProps</code> .	Server Components (RSC) secara default di sisi server.
Pengelolaan Layout	Layout manual via <code>Custom _app.js</code> atau wrapper.	Built-in nested layouts via <code>layout.tsx</code> tanpa re-render.
Pengambilan Data	Terpusat di tingkat halaman (<code>getStaticProps/gSSP</code>).	Async/Await langsung di dalam Server Component manapun.
Streaming & Suspense	Terbatas, memerlukan konfigurasi kustom rumit.	Dukungan native via <code>loading.tsx</code> dan React Suspense.
Kompatibilitas SEO	Baik, namun memerlukan penataan HEAD manual.	Built-in Metadata API dinamis & statis berkinerja tinggi.

Tabel 6.1: Matriks perbandingan teknis antara Legacy Pages Router dan Modern App Router Next.js (CloudRevol, 2026).

C. React Server Components (RSC) vs Client Components ("use client")

Salah satu inovasi terbesar dalam arsitektur Next.js App Router adalah React Server Components (RSC). Secara default, semua komponen yang dibuat di dalam direktori `app/` adalah Server Components (CloudRevol, 2026). Server Components dieksekusi eksklusif di lingkungan server Node.js atau Edge runtime selama proses pembuatan HTML, sehingga kode JavaScript komponen tersebut TIDAK DIKIRIMKAN ke browser klien.

Dampak Bundle Size Zero-KB: Eksekusi terjadi sepenuhnya di server, mengeliminasi pengiriman modul JavaScript heavy-weight ke browser.

Akses Sumber Daya Server: Dapat mengakses database, sistem file, atau API internal secara langsung tanpa membuka endpoint HTTP publik.

Keamanan Tingkat Tinggi: Kredensial seperti API Key, Database Secret, dan Token Autentikasi aman di server tanpa risiko bocor ke klien (CloudRevol, 2026).

Client Components ("use client"): Diperlukan saat komponen membutuhkan interaktivitas browser (event listener seperti `onClick`, `onChange`), React Hooks (`useState`, `useEffect`, `useReducer`), atau API browser (`localStorage`, `geolocation`). Dideklarasikan dengan instruksi `"use client"` di baris paling atas file.



PRINSIP DESAIN COMPONENT BOUNDARY: Gunakan pendekatan 'Leaf Components' saat menentukan batas `"use client"`. Biarkan hirarki komponen utama tetap menjadi Server Component, dan isolasi kebutuhan state interaktif ke dalam komponen klien sekecil mungkin di tingkat paling bawah (daun hirarki) untuk menjaga bundle size tetap minimal (CloudRevol, 2026).

D. Mekanisme Dynamic Layouts, Nested Routing, dan Parallel Routes

App Router menggunakan konvensi penamaan folder untuk menentukan rute URL. Setiap folder merepresentasikan segmen rute, dan file `page.tsx` menjadi antarmuka publik rute tersebut. Fitur unggulan dari struktur ini adalah Layouts Bersarang (Nested Layouts) melalui file `layout.tsx` (CloudRevol, 2026).

Layouts (layout.tsx): Elemen UI yang dibagikan di antara beberapa halaman (seperti Sidebar, Header, atau Footer). Saat pengguna berpindah halaman di dalam segmen rute yang sama, layout.tsx TIDAK dirender ulang (preserving state & DOM status).

Templates (template.tsx): Wrapper antarmuka yang membungkus komponen anak, namun berbedanya dengan layout, template.tsx SELALU membuat ulang instance komponen baru setiap kali rute berubah (berguna untuk animasi transisi halaman atau logging analytics).

Dynamic Routes ([slug]): Segmen dinamis yang didefinisikan dengan tanda kurung siku (misal: app/blog/[id]/page.tsx), tempat variabel id diterima secara otomatis melalui objek params komponen.

E. Streaming SSR, Suspense, dan Loading UI

Pada aplikasi SSR tradisional, server harus menunggu seluruh data selesai diambil dari database sebelum mulai mengirimkan HTML ke browser. Jika satu kueri lambat, keseluruhan halaman akan tertahan dalam kondisi layar putih (blank screen). Next.js App Router memecahkan masalah ini melalui Streaming SSR terintegrasi dengan React Suspense.

Dengan Streaming SSR, server langsung mengirimkan kerangka HTML utama (serta penampung sementara / skeleton) ke browser secara instan melalui HTTP chunked transfer encoding. Saat proses pengambilan data asinkron di server selesai, bagian komponen yang tertunda (di dalam batas `<Suspense>`) di-stream secara bertahap dan disisipkan ke halaman browser tanpa memblokir interaksi pengguna.

F. Metadata API, SEO, & Optimasi Core Web Vitals (LCP, INP, CLS)

Performa dan keterindeksan search engine merupakan indikator kunci keberhasilan aplikasi web modern. Next.js menyediakan Metadata API terpadu yang menggantikan tag `<head>` manual (CloudRevol, 2026). Metadata dapat dideklarasikan secara statis melalui objek `export const metadata: Metadata` atau secara dinamis melalui fungsi `export async function generateMetadata()`.

Selain metadata, Next.js mendukung pencapaian ambang batas Core Web Vitals berdasarkan standar Chrome UX Report (CrUX 75th percentile) (SEO-Kreativ, 2026):

LCP (Largest Contentful Paint <= 2.5s): Waktu pemuatan elemen visual terbesar di viewport. Dioptimalkan via komponen `<Image src="..." priority fetchpriority="high" />` untuk mengeliminasi lazy loading pada elemen hero image (CloudRevol, 2026; SEO-Kreativ, 2026).

INP (Interaction to Next Paint <= 200ms): Responsivitas antarmuka terhadap interaksi pengguna (menggantikan FID sejak Maret 2024). Dioptimalkan dengan meminimalkan JavaScript Long Tasks (>50ms) pada main thread serta mengisolasi state ke Client Components (SEO-Kreativ, 2026).

CLS (Cumulative Layout Shift <= 0.1): Stabilitas visual antarmuka saat pemuatan. Dioptimalkan secara otomatis melalui fungsi `next/font` yang menetapkan dimensi CSS font secara eksplisit serta penentuan width & height pada aset gambar (CloudRevol, 2026; SEO-Kreativ, 2026).

G. Route Handlers, Middleware, & Proteksi Akses Server-Side

Next.js App Router menyediakan file `route.ts` (Route Handlers) untuk membangun Web API berbasis standar objek Web Request dan Response (CloudRevol, 2026). Route Handlers mendukung method HTTP standar seperti GET, POST, PUT, PATCH, dan DELETE.

Untuk keamanan dan kendali tingkat tinggi, Next.js menyediakan file ``middleware.ts`` yang dieksekusi di server sebelum sebuah permintaan (request) diselesaikan ke rute halaman. Middleware sangat ideal untuk verifikasi token JWT/Session cookie, pengalihan rute otomatis (redirects untuk pengguna yang belum terautentikasi), manipulasi header HTTP, dan penanganan kustom kebiasaan CORS (CloudRevol, 2026).

H. Integrasi Zero-Runtime CSS & Type-Safe Data Layer (TypeScript & Zod)

Dalam ekosistem React Server Components, pustaka CSS-in-JS tradisional berbasis runtime (seperti Styled-Components dan Emotion) tidak lagi kompatibel karena membutuhkan manipulasi DOM dinamis di browser (Kanopy Labs, 2026; PkgPulse Guides, 2026). Oleh karena itu, standar styling modern beralih ke Zero-Runtime CSS engines seperti Tailwind CSS v4 (ditenagai Rust-based Oxide Engine) atau CSS Modules yang mengompilasi CSS statis di build-time (Kanopy Labs, 2026).

Di sisi integritas data, Next.js dipadukan dengan konfigurasi Strict TypeScript (``strict: true``, ``noUncheckedIndexedAccess: true``) serta pustaka validasi skema runtime ****Zod**** (Egnworks, 2026). Pola ``z.infer<typeof Schema>`` memberikan jaminan keamanan tipe data ujung-ke-ujung (end-to-end type safety) dari respon API server hingga tampilan antarmuka klien.

I. Verifikasi Kode Statis dalam Era Meta-Framework (SonarQube)

Seiring dengan meluasnya penggunaan AI Coding Assistants (seperti GitHub Copilot, ChatGPT, dan Cursor yang digunakan oleh 72% pengembang setiap hari) (SonarSource, 2026), terjadi pergeseran beban kerja pengembangan menuju 'Verification Bottleneck'. Laporan industri menunjukkan bahwa 96% pengembang tidak sepenuhnya percaya pada fungsionalitas kode buatan AI karena sering menghasilkan kode yang terlihat benar namun menyimpan bug tersembunyi atau celah keamanan (SonarSource, 2026).

Penggunaan alat analisis statis otomatis seperti ****SonarQube**** menjadi lapisan verifikasi wajib dalam pipeline CI/CD Next.js. Pengembang yang memanfaatkan SonarQube mencatatkan penurunan tingkat cacat (defects) hingga 20% dan celah keamanan (vulnerabilities) hingga 24%, sehingga mempercepat rilis aplikasi tanpa menambah akumulasi utang teknis (technical debt) (SonarSource, 2026).

4. Langkah-Langkah Praktikum Terpandu

Pada praktikum ini, mahasiswa akan membangun proyek aplikasi web penuh *****UNS Portal & Student Task Management App***** menggunakan Next.js App Router, React Server Components (RSC), Client Component Boundaries, Middleware Autentikasi, serta Metadata API SEO.

1. Langkah 1: Inisialisasi Proyek & Konfigurasi App Router Root Layout

Buat proyek Next.js baru dengan TypeScript dan Tailwind CSS, kemudian susun Root Layout pada file `app/layout.tsx` yang menyertakan optimasi font melalui `next/font` dan Metadata API SEO statis.

```
// app/layout.tsx - Root Layout dengan Metadata API & Font Optimization
import type { Metadata } from "next";
import { Inter } from "next/font/google";
import "./globals.css";

const inter = Inter({ subsets: ["latin"] });

export const metadata: Metadata = {
  title: {
    default: "UNS Portal & Task Management System",
    template: "%s | UNS TaskFlow"
  },
  description: "Sistem Manajemen Tugas Mahasiswa D3 Teknik Informatika Sekolah Vokasi UNS",
  keywords: ["UNS", "Sekolah Vokasi", "Teknik Informatika", "Next.js", "App Router"],
  authors: [{ name: "Lab RPL & Front-End Engineering SV UNS" }],
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="id">
      <body className={` ${inter.className} bg-slate-50 text-slate-900 min-h-screen antialiased`}>
        <header className="bg-navy-900 text-white border-b border-slate-700 p-4 shadow-md">
          <div className="max-w-7xl mx-auto flex justify-between items-center">
            <h1 className="text-xl font-bold tracking-tight">🎓 UNS TaskFlow App Router</h1>
            <span className="text-xs bg-blue-600 px-3 py-1 rounded-full font-medium">Next.js RSC Mode</span>
          </div>
        </header>
        <main className="max-w-7xl mx-auto p-6">{children}</main>
      </body>
    </html>
  );
}
```

```
);
}
```

2. Langkah 2: Pembuatan Nested Dashboard Layout & Streaming Loading UI

Buat struktur direktori `app/dashboard/` beserta file `layout.tsx` untuk membagikan antarmuka Sidebar, serta file `loading.tsx` untuk menampilkan skeleton secara otomatis via Streaming SSR.

```
// app/dashboard/layout.tsx - Nested Layout untuk Segmen Dashboard
import Link from "next/link";

export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div className="flex gap-6">
      <aside className="w-64 bg-white p-5 rounded-xl shadow-sm border border-slate-200 h-fit">
        <h2 className="text-xs font-semibold text-slate-400 uppercase tracking-wider mb-4">Navigasi Utama</h2>
        <nav className="flex flex-col gap-2">
          <Link href="/dashboard" className="px-3 py-2 text-sm font-medium rounded-lg bg-blue-50 text-blue-700 hover:bg-blue-100">
            📌 Overview Tugas
          </Link>
          <Link href="/dashboard/analytics" className="px-3 py-2 text-sm font-medium rounded-lg text-slate-600 hover:bg-slate-100">
            📊 Analisis Performa
          </Link>
        </nav>
      </aside>
      <section className="flex-1">{children}</section>
    </div>
  );
}

// app/dashboard/loading.tsx - Streaming Skeleton Fallback UI
export default function DashboardLoading() {
  return (
    <div className="animate-pulse space-y-4">
      <div className="h-8 bg-slate-200 rounded w-1/3"></div>
      <div className="h-32 bg-slate-200 rounded-xl"></div>
      <div className="h-64 bg-slate-200 rounded-xl"></div>
    </div>
  );
}
```

3. Langkah 3: Implementation of React Server Component (RSC) Data Fetching

Buat halaman utama dashboard pada `app/dashboard/page.tsx` sebagai **React Server Component**. Komponen ini melakukan data fetching async langsung di server tanpa memerlukan `useState/useEffect`.

```
// app/dashboard/page.tsx - React Server Component (Zero Bundle JS sent to client)
import { Suspense } from "react";
import TaskListFetcher from "@components/TaskListFetcher";
import TaskFormClient from "@components/TaskFormClient";

export const metadata = {
  title: "Dashboard Tugas Mahasiswa",
};

export default async function DashboardPage() {
  return (
    <div className="space-y-6">
      <div>
        <h2 className="text-2xl font-bold text-slate-800">Daftar Tugas Praktikum
aktif</h2>
        <p className="text-sm text-slate-500">Dirender langsung dari server
menggunakan React Server Components (RSC).</p>
      </div>

      {/* Interactive Client Component Isolated */}
      <TaskFormClient />

      {/* Streaming Async Server Component with Suspense Boundary */}
      <Suspense fallback={<div className="p-4 bg-yellow-50 text-yellow-800 rounded-
lg">⌚ Mengambil data dari database server...</div>}>
        <TaskListFetcher />
      </Suspense>
    </div>
  );
}
```

4. Langkah 4: Isolasi Interactive Client Component ("use client") & Zod Validation

Buat komponen form interaktif pada `components/TaskFormClient.tsx`. Tambahkan instruksi `"use client"` di baris paling atas karena komponen ini menangani event input pengguna dan validasi Zod.

```
// components/TaskFormClient.tsx - Client Component Boundary
"use client";

import { useState } from "react";
import { z } from "zod";

const TaskSchema = z.object({
  title: z.string().min(3, "Judul tugas minimal 3 karakter"),
});
```

```

    subject: z.string().min(2, "Mata kuliah wajib diisi"),
    priority: z.enum(["low", "medium", "high"]),
  });

export default function TaskFormClient() {
  const [formData, setFormData] = useState({ title: "", subject: "", priority:
"medium" });
  const [errors, setErrors] = useState<string | null>(null);

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    const result = TaskSchema.safeParse(formData);
    if (!result.success) {
      setErrors(result.error.errors[0].message);
      return;
    }
    setErrors(null);
    alert(`[Klien Validated] Tugas "${formData.title}" siap dikirim ke Server
Action!`);
    setFormData({ title: "", subject: "", priority: "medium" });
  };

  return (
    <form onSubmit={handleSubmit} className="bg-white p-5 rounded-xl border border-
slate-200 space-y-4 shadow-sm">
      <h3 className="font-semibold text-slate-700">Tambah Tugas Baru (Client
Component)</h3>
      {errors && <div className="p-3 bg-red-50 text-red-600 text-xs rounded-
lg">{errors}</div>}
      <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
        <input
          type="text"
          placeholder="Judul Tugas..."
          value={formData.title}
          onChange={(e) => setFormData({ ...formData, title: e.target.value })}
          className="px-3 py-2 border rounded-lg text-sm"
        />
        <input
          type="text"
          placeholder="Mata Kuliah..."
          value={formData.subject}
          onChange={(e) => setFormData({ ...formData, subject: e.target.value })}
          className="px-3 py-2 border rounded-lg text-sm"
        />
        <button type="submit" className="bg-blue-600 text-white font-medium px-4 py-2
rounded-lg text-sm hover:bg-blue-700">
          + Simpan Tugas
        </button>
      </div>
    </form>
  );
}

```

5. Langkah 5: Konfigurasi Middleware Proteksi Rute (`middleware.ts`)

Tambahkan file `middleware.ts` di akar direktori proyek untuk memeriksa ketersediaan token autentikasi sebelum mengizinkan akses ke area `/dashboard`.

```
// middleware.ts - Server-Side Access Control & Route Guard
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
  const authToken = request.cookies.get("uns_session")?.value;
  const isDashboardRoute = request.nextUrl.pathname.startsWith("/dashboard");

  if (isDashboardRoute && !authToken) {
    // Redirect otomatis ke halaman login jika token tidak ditemukan
    return NextResponse.redirect(new URL("/login?auth_error=1", request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/dashboard/:path*"],
};
```

6. Tugas Praktikum Mandiri (Case Project Mahasiswa)

Setiap mahasiswa wajib mengembangkan antarmuka aplikasi web berbasis Next.js App Router & RSC sesuai dengan topik proyek individu dan dokumen Software Requirements Specification (SRS) masing-masing. Aplikasi yang dibuat wajib mengimplementasikan 7 Komponen Spesifik Arsitektur App Router berikut:

- Struktur App Router File-Based Routing:** Mengatur struktur folder direktori app/ yang mencakup minimal 3 segmen rute publik/privat serta penggunaan file konvensi (layout.tsx, page.tsx, loading.tsx, error.tsx).
- Penerapan Dominan React Server Components (RSC):** Minimal 70% dari komponen UI dibuat sebagai React Server Components (RSC) tanpa instruksi "use client" untuk mengoptimalkan performa dan meminimalkan JS bundle size.
- Isolasi Komponen Klien ("use client") & Zod:** Mengisolasi kebutuhan event listener/state interaktif ke dalam Client Components secara ketat pada bagian 'daun' hirarki UI.
- Nested Layouts & Shared UI Preserving:** Menyusun minimal 2 tingkat layout bersarang (Root Layout & Sub-Dashboard Layout) yang mempertahankan state UI saat navigasi antar rute.
- Streaming SSR & Loading Skeleton:** Menggunakan fitur `` dan file `loading.tsx` untuk menampilkan skeleton placeholder saat proses pengambilan data asinkron berjalan.

- f) **Middleware Proteksi Rute & Security:** Mengkonfigurasi `middleware.ts` untuk memvalidasi cookie session/token autentikasi sebelum mengizinkan rute tertentu, serta menyediakan pengalihan otomatis.
- g) **Metadata API Dynamic SEO:** Menyediakan objek Metadata API statis dan dinamis (`generateMetadata`) pada setiap halaman untuk keterindeksan SEO dan ketercapaian Core Web Vitals.

Rubrik Penilaian Praktikum Bab VI

Kriteria Penilaian	Bobot	Indikator Kelayakan & Keunggulan Teknikal
Arsitektur App Router & RSC	30%	Kesesuaian struktur app/, isolasi Client Components, dominasi React Server Components, dan penggunaan file konvensi secara tepat.
Data Fetching & Streaming SSR	20%	Implementasi async server-side data fetching, penggunaan <code>Suspense</code> , <code>loading.tsx</code> skeleton, dan penanganan caching.
Middleware & Access Control	20%	Keandalan <code>middleware.ts</code> dalam mencegah rute privat, pemeriksaan cookie/session token, dan penanganan redirect.
SEO & Metadata API	15%	Pengaturan Metadata API dinamis/statis, optimasi aset gambar/font, dan kepatuhan terhadap Core Web Vitals.
Kesesuaian SRS & Matriks UI	15%	Kelengkapan fitur antarmuka sesuai dokumen SRS proyek mahasiswa serta kerapian dokumentasi matriks pemetaan.

Tabel 6.2: Rubrik dan kriteria penilaian praktikum Bab VI (Skala Total 100%).

Daftar Pustaka & Referensi Akademis-Industri Terverifikasi

CloudRevol. (2026). Top 20 Next.js Best Practices Every Developer Should Follow in 2026. CloudRevol Development Insights Report.

Egnworks. (2026). Advanced TypeScript Patterns Every Frontend Developer Should Know in 2026. Egnworks Engineering Lab.

InfoQ. (2026). State of JavaScript 2025: Survey Reveals a Maturing Ecosystem with TypeScript Cementing Dominance. InfoQ Web Development Trend.

Kanopy Labs. (2026). Panda CSS vs StyleX vs Tailwind v4: CSS Performance in 2026. Kanopy Technology Guides.

PkgPulse Guides. (2026). The State of CSS-in-JS in 2026: Is It Dead? PkgPulse Ecosystem Analytics.

SEO-Kreativ. (2026). Core Web Vitals: Optimizing LCP, INP & CLS [Guide 2026]. Technical SEO & Web Performance Research.

SonarSource. (2026). State of Code Developer Survey Report 2026: The Verification Bottleneck in AI-Assisted Engineering.

SVAR UI. (2026). Is React Still the King in 2026? React vs Vue vs Svelte for Modern Web Apps. SVAR Engineering Blog.

Tien Nguyen. (2026). Zustand vs Redux: Which State Manager Should You Use in 2026? Tien Nguyen Craft Series.